

Pydantic x Modal / October 2026

Why even have an SDK?

Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic**
- Founder & CEO of **Pydantic Labs**

The Pydantic Stack

- **Pydantic Validation**: contracts for data
- **Pydantic AI**: production grade agent framework
- **Pydantic Logfire**: observability, evals, prompt management, LLMOps
- **Pydantic AI Gateway**: one endpoint, every model
- **Pydantic Monty**: Python Sandbox

**OpenTelemetry already exists.
Coding agents exist.**

So why did we write three SDKs and a CLI?

What OpenTelemetry gives you

- **A wire format** - OTLP, plus semantic conventions for what to call things
- **Instrumentation for everything** - `opentelemetry-instrumentation-fastapi`, `-httpx`, `-asyncpg`, ~100 more
- **Any backend** - Jaeger, Tempo, Datadog, Logfire, a file

What it doesn't give you

- **Defaults.** You assemble the provider, processor, exporter and resource yourself, ~40 lines before the first span
- **A span before it ends.** Exporters only receive finished spans, so a running request is invisible
- **Structure for values.** An attribute is a string, number, bool, or a flat list of those

Two lines

```
import logfire
from fastapi import FastAPI

app = FastAPI()

logfire.configure()
logfire.instrument_fastapi(app)
```

- `instrument_fastapi` calls `opentelemetry-instrumentation-fastapi`, then adds the message template and span-type attributes described next
- 34 `instrument_*` methods built the same way: Pydantic AI, OpenAI, Anthropic, httpx, asyncpg, Django, Celery, Redis, MCP, Monty, ...
- `logfire.configure()` reads the token from its argument, `LOGFIRE_TOKEN`, or `.logfire/`, in that order

Three things the SDK adds that the backend relies on

message templates · value schemas · pending spans

Opinion 1: the template is the name

```
logfire.info('hello {name}', name=user)

# span name           'hello {name}'
# logfire.msg_template 'hello {name}'
# logfire.msg         "hello name=User(id=1, email='x@y.z', signed_up=datetime(...))"
# logfire.json_schema  {"properties": {"name": {"x-python-datatype": "PydanticModel", ...}}}
```

- The **template** is the span name: low cardinality, so the UI can group, count and alert on it
- The **message** is the template with values substituted, for reading
- f-strings work too: `logfire.info(f'hello {user.email}')` - `executing` finds the call's AST and recovers the template
- Values are sent as ordinary OTel attributes; the schema records the Python types that JSON drops

Opinion 2: values look like Python

What the wire carries

```
{"id": 1, "email": "x@y.z",  
  "signed_up": "2026-01-01T09:30:00Z"}
```

plus, once per attribute, in `logfire.json_schema` :

```
{"title": "User",  
  "x-python-datatype": "PydanticModel",  
  "properties": {"signed_up": {  
    "type": "string", "format": "date-time",  
    "x-python-datatype": "datetime"}}}
```

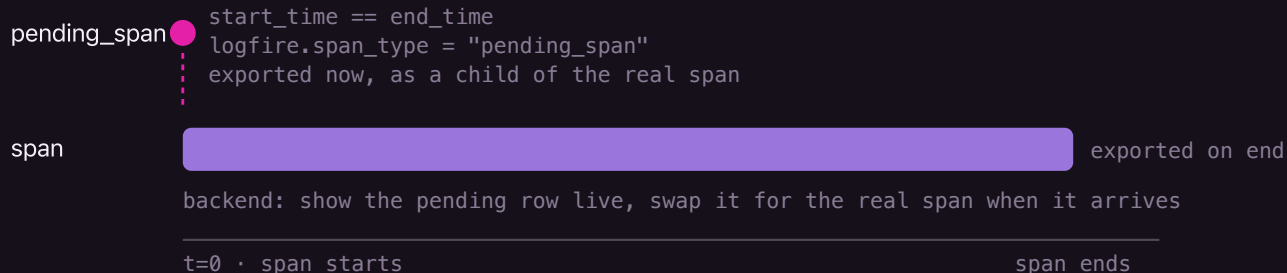
What you see

```
User(  
    id=1,  
    email='x@y.z',  
    signed_up=datetime(2026, 1, 1, 9, 30),  
)
```

- `x-python-datatype` for models, dataclasses, `datetime`, `Decimal`, `UUID`, `Enum`, `Path`, exceptions
- DataFrames carry `x-columns`, `x-row-count`, `x-shape`
- Plain `str`, `int`, `float`, `bool` need nothing: `{}`

Opinion 3: pending spans

- OTLP has no "span started" message. Exporters run on `on_end`, so a 30-second agent run is invisible for 30 seconds
- When a span starts, the SDK exports a **zero-duration copy** of it, tagged so the backend can replace it later



- `RemovePendingSpansExporter` drops the copy when the real span is in the same export batch, so short spans are sent once
- Only the Logfire exporter receives them; `additional_span_processors` do not

Same opinions, three languages

	PYTHON LOGFIRE-SDK 6	JS / TS LOGFIRE 0.9	RUST LOGFIRE 0.12
Built on	opentelemetry-sdk	@opentelemetry/sdk-node	tracing + opentelemetry
Auto-instrumentation	34 instrument_* (OTel contrib)	node auto-instrumentations	anything using tracing
logfire.msg + msg_template	✓	✓	msg; template = span name
logfire.json_schema	✓	✓	✓
Pending spans	✓	✗	✓
Prompts / variables	✓	OpenFeature / OFREP	OpenFeature / OFREP

Hello, TypeScript

```
import * as logfire from 'logfire'

logfire.configure({ serviceName: 'weather-app' })

logfire.span('forecast for {city}', { city: 'Paris', units: 'C' }, {}, async () => {
  const forecast = await fetchForecast('Paris')
  logfire.info('got {days} days', { days: forecast.length, forecast })
})
```

- `npm i logfire` - wraps `@opentelemetry/sdk-node` and Node auto-instrumentations
- `@pydantic/logfire-api` (zero deps, for libraries), `-browser`, `-cf-workers`
- Same `logfire.msg_template` / `logfire.msg` / `logfire.json_schema` attributes

Hello, Rust

```
fn main() -> Result<(), Box<dyn std::error::Error>> {  
    let logfire = logfire::configure().finish()?;  
    let _guard = logfire.shutdown_guard();  
  
    logfire::span!("forecast for {city}", city = "Paris").in_scope(|| {  
        logfire::info!("got {days} days", days = 7u64);  
    });  
    Ok(())  
}
```

- `cargo add logfire` - a `tracing` subscriber layer over `opentelemetry`, so existing `tracing` spans and events are exported unchanged
- Pending spans included; the macros set `logfire.msg` and `logfire.json_schema`
- The CLI is built on the same crate

Coding agents are the SDK's second audience

`configure()` **plus one** `instrument_*` **call per library is an API a model gets right. 40 lines of OTel setup is not.**

Demo: one trace, from agent to app

```
logfire.configure(service_name='weather-app')
logfire.instrument_pydantic_ai()
logfire.instrument_monty()

app = FastAPI()
logfire.instrument_fastapi(app)

@app.get('/weather')
async def weather(q: str) -> dict[str, str]:
    result = await weather_agent.run(f"What's the weather in {q}, in Celsius?")
    return {'answer': result.output}
```

- One trace: HTTP request, Pydantic AI run, LLM call, Monty codemode execution, tool calls
- Pending spans: the trace is on screen before the model has answered

Prompts are variables

```
prompt = logfire.template_var(  
    name='prompt__weather_agent',  
    type=str,  
    default='Combine tool calls into a single code execution call. Answer in {{units}}.',  
    inputs_type=WeatherInputs, # a BaseModel with `units: str`  
)  
  
with prompt.get(WeatherInputs(units='celsius'), label='production') as resolved:  
    result = await weather_agent.run(question, instructions=resolved.value)
```

- A prompt is a managed variable named `prompt__<slug>`, with the same versions, targeting rules and rollouts as a feature flag
- Handlebars `{{placeholders}}`; `@{other_variable}@` inserts another variable
- Python uses the `logfire[variables]` extra. Other languages: Logfire serves **OFREP**, so any **OpenFeature** client can evaluate the same variables

One binary, every ecosystem

```
➤ uv tool install logfire-cli # or: uvx logfire-cli  
➤ npm install --global logfire-cli # or: npx logfire-cli  
➤ pip install logfire # logfire 6 = logfire-sdk + logfire-cli
```

- One Rust binary. PyPI wheels for 17 targets, npm for 8. Neither Rust nor Python is needed to run it
- **For people:** `logfire signup`, `auth`, `init`, `project current`, `token read`
- **For scripts:** `--output json|jsonl`, `--no-input`, documented exit codes
- **For agents:** `logfire wizard` starts Claude Code or Codex in your repo with the Logfire setup skill; `logfire mcp ...` calls the hosted MCP server

Demo: auth + query

```
logfire auth
logfire mcp projects

logfire mcp query run \
  'SELECT span_name, duration, message FROM records ORDER BY start_timestamp DESC LIMIT 20' \
  --project demo

logfire --output json mcp query run --file slow.sql --project demo
```

- The query runs through the hosted MCP server, the same tool a coding agent calls
- Same binary and login for people, CI and agents; `--output json` for the last two

Thank you

Thanks to **Modal** for hosting



pydantic.dev/contact

@samuelcolvin · @pydantic